| EXPT NO: 2 | A PYTHON PROGRAM TO IMPLEMENT THE ASSIGNMENT PROBLEM |
|---|---|
| DATE: | |

**AIM:**

To understand and solve the assignment problem using python, assignment team of workers.

**PROCEDURE:**

1. INPUT DATA: Define the cost matrix where each element represents the cost of assigning a particular task to a specific worker.

2. ALGORITHM: Use an optimization algorithm like the Hungarian algorithm (or the Munkres algorithm) to find the optimal assignment that minimizes the total cost.

3. OUTPUT: Display the optimal assignment along with the minimum total cost.

**PROBLEM 1 : SOLVE THE ASSIGNMENT PROBLEM OF 4 JOBS AND 4 WORKERS FOR THE FOLLOWING MATRIX**

| | | | |
|---|---|---|---|
| 1 | 4 | 6 | 3 |
| 7 | 10 | 9 | 4 |
| 5 | 11 | 7 | 8 |
| 7 | 8 | 5 | 3 |

**CODE**

```
import numpy as np

from scipy.optimize import linear_sum_assignment

def solve_assignment_problem(cost_matrix, problem_type="balanced"):

    original_cost_matrix = cost_matrix.copy()

    if problem_type == "unbalanced":

        print("Unbalanced assignment problem detected.")

         num_workers, num_tasks = cost_matrix.shape

        max_dim = max(num_workers, num_tasks)
```

```
        padded_cost_matrix = np.zeros((max_dim, max_dim))
        padded_cost_matrix[:num_workers, :num_tasks] = cost_matrix

        print("Padded cost matrix:")
        print(padded_cost_matrix)

        cost_matrix = padded_cost_matrix
    else:
        num_workers, num_tasks = cost_matrix.shape
    row_ind, col_ind = linear_sum_assignment(cost_matrix)

    if problem_type == "unbalanced":
        valid = (row_ind < original_cost_matrix.shape[0]) & \
                (col_ind < original_cost_matrix.shape[1])

        row_ind = row_ind[valid]
        col_ind = col_ind[valid]

        total_cost = original_cost_matrix[row_ind, col_ind].sum()
    else:
        total_cost = cost_matrix[row_ind, col_ind].sum()
    return row_ind, col_ind, total_cost
cost_matrix = np.array([
    [1, 4, 6, 3],
    [9, 7, 10, 9],
    [4, 5, 11, 7],
    [8, 7, 8, 5]
```

```
])
row_ind, col_ind, total_cost = solve_assignment_problem(cost_matrix, "balanced")
print("\nOptimal Assignment:")
for i in range(len(row_ind)):
    print(f"Job {row_ind[i]+1} assigned to Machine {col_ind[i]+1}")
print(f"\nMinimum Total Cost: {total_cost}")
```

**OUTPUT**

Optimal Assignment:

Job 1 assigned to Machine 1

Job 2 assigned to Machine 3

Job 3 assigned to Machine 2

Job 4 assigned to Machine 4

Minimum Total Cost: 21.0

**PROBLEM 2 : SOLVE THE ASSIGNMENT PROBLEM OF 4 JOBS AND 4 WORKERS FOR THE FOLLOWING MATRIX**

```
[10  2  9]
[7  5  6]
[8  1  4]
```

**CODE**

```
 import numpy as np
from scipy.optimize import linear_sum_assignment
def solve_assignment_problem(cost_matrix):
    row_ind, col_ind = linear_sum_assignment(cost_matrix)
    total_cost = cost_matrix[row_ind, col_ind].sum()
    return row_ind, col_ind, total_cost
cost_matrix = np.array([
    [10, 2, 9],
    [7, 5, 6],
    [8, 1, 4]
```

```
])
row_ind, col_ind, total_cost = solve_assignment_problem(cost_matrix)
print("\nOptimal Assignment:")
for i in range(len(row_ind)):
    print(f"Job {row_ind[i]+1} assigned to Machine {col_ind[i]+1}")
print(f"\nMinimum Total Cost: {total_cost}")
```

**OUTPUT**

```
Optimal Assignment:
Job 1 assigned to Machine 2
Job 2 assigned to Machine 1
Job 3 assigned to Machine 3
Minimum Total Cost: 13
```

**RESULT:**

Thus Understanding And Solving Of The Assignment Problem Using Python, Assignment Team Of Workers.